Bazzite Unified
Home Server + Gaming Station

SSH-first · Zero New Open Ports · Headless by Default

bazzite-deck · Docker · firewalld · Fail2ban · Nginx Proxy Manager

Nextcloud AIO · DuckDNS · WireGuard VPN · Sunshine/Moonlight

SELinux · systemd-run · gamescope-session-steam

A single, self-contained guide building a headless home server with two on-demand graphical sessions — KDE Plasma and Gaming Mode — from a single bazzite-deck installation.

Updated March 2026 · All phases additive and ordered · No rebase required

Table of Contents

OVERVIEW	**What You Get & Architecture**
Phase 0	**OS Installation**
Phase 1	**First Boot: SSH Bootstrap & Headless Configuration**
Phase 2	**Router & Network Preparation**
Phase 3	**System Package Installation**
Phase 4	**Ironclad Security — SELinux · SSH Keys · firewalld · Fail2ban**
Phase 5	**Docker Initialisation**
Phase 6	**Data Drive Setup**
Phase 7	**Deploying Services — DuckDNS · NPM · Nextcloud AIO**
Phase 8	**Session Management Scripts**
Phase 9	**WireGuard VPN + Sunshine/Moonlight Streaming**
Phase 10	**Link & Initialise — NPM config · Nextcloud setup**
Phase 11	**Testing & Backups**
Appendix A	Troubleshooting
Appendix B	Normal Workflows (Quick Reference)
Appendix C	Optional: AMD GPU Hardware Acceleration for Nextcloud
Appendix D	OS Updates, Rollbacks & Container Updates
Appendix E	Server Health Monitor

OVERVIEW — What You Get & Architecture

This guide turns a single desktop PC into two things at once: a 24/7 home server running self-hosted cloud software, and an on-demand gaming station. The machine normally sits at a blank, idle login screen. You trigger either KDE Desktop or Steam Gaming Mode remotely over SSH whenever you want them — the machine returns to idle automatically. All server software runs in isolated Docker containers in the background, independent of any graphical session.

Components

Component	What You Get
Headless server (default)	Boots to idle SDDM login screen. Docker containers run as systemd services. SSH always on port 22022.
KDE Wayland session	Full KDE Plasma on the physical display, on demand via SSH. Required to launch Sunshine for streaming.
Gaming Mode session	Steam Deck-style gamescope session. Direct GPU scanout, controller-native. Refused if another user session is active.
WireGuard VPN + Sunshine	Hardware-accelerated game and desktop streaming over an encrypted VPN tunnel. VA-API hardware encode, UDP transport. Moonlight is the client. No RDP.
Nextcloud AIO	Self-hosted cloud: files, calendar, contacts. Behind Nginx Proxy Manager with Let's Encrypt via DuckDNS.
SSH-first design	Every operation is a single SSH command. No ports beyond 22022 and 51820/UDP are ever opened to the public internet.

Recommended Minimum Hardware

Component	Minimum / Recommended
CPU	4-core x86_64 minimum · 6-core+ recommended
RAM	8 GB DDR4 minimum · 16 GB+ recommended (Fulltextsearch requires 16 GB+)
OS Drive	120 GB NVMe SSD minimum · 500 GB+ NVMe recommended
Data Drive	Any size HDD or SSD · Dedicated, mounted at /srv/nextcloud-data
GPU	AMD recommended (plug-and-play) · NVIDIA: beta for Gaming Mode only, not VA-API
Network	Gigabit Ethernet · Wi-Fi strongly discouraged for servers
Display (headless)	HDMI or DisplayPort dummy plug ($3–5) — REQUIRED for headless GPU init. Without a physical display signal the GPU will not initialise its output, preventing KDE and gamescope from starting.

Action Flow

Action	Flow
Launch KDE	SSH → sudo start-kde.sh → writes one-shot SDDM autologin → restarts SDDM → KDE Wayland appears → systemd-run timer restores headless config 10 s later.
Game streaming	SSH → sudo start-kde.sh → activate WireGuard client → open Moonlight → connect to 10.8.0.1. Or use start-gaming.sh for Steam Big Picture streaming.
Gaming Mode (local)	SSH → sudo start-gaming.sh → checks seat0 → writes one-shot autologin → gamescope session appears. Steam Power menu exits to idle SDDM.

Critical Pitfalls & How This Guide Avoids Them

#	Description
#1	bazzite-deck boots to Gaming Mode on first install — enter Desktop Mode, enable SSH, apply the headless SDDM override. All in one physical session.
#2	SSH is not enabled by default on Bazzite — Phase 1 Step 2 enables sshd immediately.
#3	SDDM reads config once per start — start scripts write a one-shot autologin config, restart SDDM, then a systemd-run transient timer restores headless config 10 s later (immune to cgroup cleanup).
#4	Sunshine listens on ALL interfaces when firewalld ports are open — Sunshine ports must NEVER be forwarded at the router. All remote streaming travels exclusively through the WireGuard tunnel.
#5	WireGuard port 51820/UDP is a Docker published port — Docker's nftables injection opens it at the kernel level, bypassing firewalld. Never add a firewalld rule for 51820/UDP.
#6	Sudoers entries require absolute paths and root-owned scripts — this guide uses /usr/local/bin/ with root:root ownership.
#7	SELinux blocks sshd on non-standard ports — semanage port must label 22022 BEFORE restarting sshd.
#8	Package installation must precede security hardening — all packages installed in Phase 3 before hardening.
#9	Auto-suspend silently kills all containers — Phase 1 Step 3 permanently masks all suspend targets.
#10	Docker bypasses firewalld for published ports (80, 443, 51820/UDP) via direct nftables injection. These ports ARE open even though firewall-cmd --list-all does not show them.
#11	Missing nofail on the secondary drive causes systemd to drop into emergency mode on failure. The fstab entry in Phase 6 always uses defaults,nofail.
#12	KillUserProcesses=yes in systemd-logind kills background subshells when the SSH session ends. Start scripts use systemd-run to schedule headless restoration as a system-level transient service — immune to cgroup cleanup.
#13	Blind mount on data drive failure: with nofail, if the drive is absent /srv/nextcloud-data becomes an empty folder on the OS drive. Fix: sudo chattr +i /srv/nextcloud-data while unmounted.
#14	The gamescope-session-plus.desktop session file was renamed to gamescope-session-steam.desktop in current Bazzite builds. The start-gaming.sh script auto-lists available sessions if the name is not found.
#15	Without an HDMI/DP dummy plug, AMD GPUs may not initialise a display output. KDE Plasma and gamescope both require an active display output to start. A $3–5 dummy plug is mandatory for headless operation.
#16	loginctl enable-linger and usermod group changes (input, video, render) require a full reboot to take effect. Sunshine will not start as a user service until after reboot.
#17	firewall-cmd --reload wipes Docker's nftables rules. Always run sudo systemctl restart docker immediately after any firewalld reload. Failing to do this takes ports 80/443 offline with no error message.

PHASE 0

OS Installation

bazzite-deck from ISO · Done physically at the machine

CONCEPT — WHY BAZZITE-DECK SPECIFICALLY?

Bazzite has two variants: standard desktop, and bazzite-deck — the HTPC/living-room variant. The -deck suffix means it ships both gamescope-session-steam (Steam Deck-style Gaming Mode) and KDE Plasma in a single signed image. Starting with bazzite-deck eliminates a separate rebase step and ensures Gaming Mode is part of the verified OS image.

1 Download the correct ISO

Go to **bazzite.gg** and use the Image Picker tool. Select: Hardware = Desktop/HTPC, GPU = AMD/Intel or NVIDIA, Desktop = KDE Plasma, and enable Steam Gaming Mode. Download the ISO and flash it to a USB drive using Fedora Media Writer or Balena Etcher.

2 Install Bazzite

Boot from the USB drive and follow the Anaconda installer. Set your username, password, hostname (under 20 characters, no spaces), timezone, and keyboard layout. Install to your OS drive. Network is not required during installation.

WARNING

Dual Boot Warning: If you have a Windows drive in the same machine, physically disconnect it before installing Bazzite to prevent accidental data loss or bootloader modification.

CONCEPT — WHY KEEP THE HOSTNAME UNDER 20 CHARACTERS?

Very long hostnames can cause issues with some older network tools and SDDM session management. Keep it short and descriptive, e.g., homeserver or bazzite-server.

3 Note your GPU type for later reference

AMD and Intel GPUs are fully supported in Gaming Mode and for VA-API hardware acceleration (Appendix C and Phase 9). NVIDIA support in Gaming Mode is available but described as beta. NVIDIA GPUs are not compatible with VA-API hardware encoding for Sunshine.

PHASE 1

First Boot: SSH Bootstrap & Headless Server Configuration

CRITICAL

Do ALL steps in this phase in sequence before rebooting. The headless SDDM override in Step 4 must be in place before your next reboot. If you reboot before completing Step 4, the machine will boot to Gaming Mode again — simply repeat this phase.

1 Enter KDE Desktop Mode from Gaming Mode

You will see Steam Big Picture on your TV or monitor. Navigate to the Power menu (bottom-right power icon) and select **Switch to Desktop**. KDE Plasma will launch on the physical display.

CONCEPT — WHAT IS KDE PLASMA?

KDE Plasma is a full desktop environment — the graphical workspace with a taskbar, file manager, application menu, and system settings. On bazzite-deck, it coexists with Gaming Mode. For server setup, we need KDE's terminal (Konsole) to run commands with keyboard input.

2 Open Konsole and enable SSH

CONCEPT — WHAT IS SSH?

SSH (Secure Shell) is an encrypted protocol for remotely accessing a computer over a network. Once enabled, you can type commands on your server from your personal computer without ever touching the server's keyboard or monitor again. The SSH daemon (sshd) is the background service that listens for incoming connections.

```
# Enable and start the SSH daemon.
# ■ sudo: run as root. ■ systemctl: manage systemd services.
# ■ enable: start automatically on every future boot. ■ --now: start immediately.
sudo systemctl enable --now sshd

# Find your active network interface and local IP address.
# Look for: dev <interface> src <your-IP> e.g., dev enp3s0 src 192.168.1.50
ip route get 1.1.1.1

# Alternative if your LAN has no internet access at setup time:
ip a

# Write down the IP address — you will use it for every SSH connection in this guide.
```

3 Disable auto-suspend

CRITICAL for servers

Bazzite may suspend the machine after inactivity, silently killing all Docker containers. A server must stay on 24/7. Permanently disable all suspend modes using two independent methods.

CONCEPT — WHAT IS "MASKING" A SYSTEMD TARGET?

systemctl disable prevents a service from starting automatically but still allows manual start. systemctl mask is stronger — it replaces the service file with a symlink to /dev/null, making it impossible to start by any means until unmasked.

Step A — KDE System Settings: System Settings → Power Management → Energy Saving → On AC Power → set "Suspend session" to Never. Click Apply.

Step B — Mask suspend targets at the OS level (authoritative method):

```
sudo systemctl mask sleep.target suspend.target \
    hibernate.target hybrid-sleep.target suspend-then-hibernate.target

# Verify all five are masked:
systemctl status sleep.target suspend.target \
    hibernate.target hybrid-sleep.target suspend-then-hibernate.target \
    | grep -E "Loaded:|●"
# Expected: every entry shows "masked" in the Loaded: line.
```

> **NOTE**
>
> systemd-remount-fs.service may show as failed on every boot on current Bazzite builds. This is a known harmless issue caused by composefs preventing remount of the immutable root. Mask it to keep a clean system state: sudo systemctl mask systemd-remount-fs.service && sudo systemctl reset-failed

4 Apply the headless SDDM override

THE MOST CRITICAL STEP

> **CONCEPT — WHAT IS SDDM?**
>
> SDDM (Simple Desktop Display Manager) is the login screen manager. On bazzite-deck, it is pre-configured to autologin into Gaming Mode on every boot. We change this for a server: we want SDDM to start but show an idle blank login screen, with no automatic login into any session.

> **CONCEPT — WHAT IS A DROP-IN CONFIG FILE?**
>
> Instead of editing a single monolithic config file (which OS updates can overwrite), SDDM reads config files from /etc/sddm.conf.d/ in alphabetical order. By naming our file zzz-headless-server.conf, the zzz- prefix guarantees it is processed absolutely last, so our empty User= and Session= fields win unconditionally.

```
sudo mkdir -p /etc/sddm.conf.d
sudo tee /etc/sddm.conf.d/zzz-headless-server.conf > /dev/null << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF
cat /etc/sddm.conf.d/zzz-headless-server.conf
sudo systemctl restart sddm
```

5 Verify the headless config survives a reboot

```
sudo systemctl reboot
# Wait ~60 seconds. Physical display should show SDDM login screen — NOT Gaming Mode.
ssh username@192.168.1.50
loginctl list-sessions
# Expected: empty output, or only your SSH session with no seat0 column entry.
```

WARNING

If you cannot SSH in on port 22: Boot back to Desktop Mode via Gaming Mode Power menu → Switch to Desktop, re-run Step 2, then Step 5.

PHASE 2

Router & Network Preparation

Done on your personal computer · Requires router admin access

1 Assign a static local IP (DHCP reservation)

CONCEPT — WHY A STATIC IP?

By default, routers hand out IP addresses automatically (DHCP). These addresses can change at any time. A DHCP reservation tells the router: "always give this specific network card (identified by MAC address) this specific IP." The server still gets its IP from DHCP — the address is just guaranteed to never change.

In your router's admin panel (typically 192.168.1.1 or 192.168.0.1): find the DHCP section, add a reservation for your server's MAC address, and assign the IP you noted in Phase 1 Step 2.

2 Set up port forwarding

CONCEPT — WHAT IS PORT FORWARDING?

Your router has one public IP address and shares it among all devices via NAT. Port forwarding creates a permanent rule: "when traffic arrives from the internet on port X, send it to the server." We open only the specific ports that need to be publicly reachable.

Port / Protocol	Purpose
80/TCP	Required by Let's Encrypt for SSL certificate issuance (HTTP-01 challenge).
443/TCP	Your Nextcloud's public HTTPS entry point. All user traffic uses this port.
22022/TCP	SSH — our custom hardened port (configured in Phase 4).
51820/UDP	WireGuard VPN — the only internet-facing port added in Phase 9. Opens the secure tunnel for Sunshine game streaming and remote access.
3478/ TCP+UDP	Required ONLY if you plan to enable Nextcloud Talk (video calls) later.

PITFALL

NEVER forward Sunshine ports (47984, 47989, 47998, 47999, 48000, 48002, 48010) at the router. These are LAN-only. Sunshine is only reachable remotely via the WireGuard VPN tunnel.

INFO

Ports 80, 443, and 51820/UDP are managed by Docker via direct nftables injection, bypassing firewalld. firewall-cmd --list-all will NOT show them as open. This is expected.

3 Create your DuckDNS account

CONCEPT — WHAT IS DUCKDNS?

Your home internet connection has a public IP that can change at any time. DuckDNS is a free dynamic DNS service: it gives you a permanent domain name (e.g., yourname.duckdns.org) and provides a tiny updater program that runs on your server. Whenever your public IP changes, the updater notifies DuckDNS and the domain name is updated within 60 seconds.

Go to **duckdns.org** and sign in with Google or GitHub. Create a free subdomain (e.g., yourname.duckdns.org) and copy your Token. Note both values — you will need them in Phase 7.

PHASE 3

System Package Installation

Docker · Fail2ban · SELinux tools · Runs on port 22 before SSH hardening

CONCEPT — WHAT IS RPM-OSTREE AND THE IMMUTABLE OS?

Bazzite is built on Fedora Atomic — an "immutable" operating system. The root filesystem is read-only. OS updates are atomic: a new OS image is downloaded in the background and applied in full on the next reboot. You cannot install packages the traditional way (dnf install). Instead, rpm-ostree install "layers" packages on top of the immutable base. A reboot is required to activate any newly layered packages.

1 OS updates — no action needed

Bazzite updates arrive as complete new images staged in the background. If an OS update ever causes an issue, one command returns to the previous working state:

```
rpm-ostree rollback && systemctl reboot
```

2 Add the Docker repository

CONCEPT — WHY DOCKER CE AND NOT PODMAN?

Bazzite comes with Podman pre-installed. However, Nextcloud AIO's mastercontainer manages multiple child containers by connecting directly to the Docker API socket. This orchestration logic requires a native Docker API; Podman's Docker compatibility layer is insufficient for AIO's needs. We install Docker CE (Community Edition).

```
sudo curl -fsSL https://download.docker.com/linux/fedora/docker-ce.repo \
    -o /etc/yum.repos.d/docker-ce.repo
```

3 Install all required packages

one reboot required

CONCEPT — WHAT IS THE PODMAN-DOCKER SHIM?

Bazzite may have a package called podman-docker installed. This shim provides a /usr/bin/docker symlink pointing to Podman. If we install Docker CE while this shim is present, both packages try to own /usr/bin/docker and conflict. We detect whether the shim is present and remove it atomically in the same operation as installing Docker CE.

```
if rpm -q podman-docker >/dev/null 2>&1; then
    rpm-ostree override remove podman-docker \
        --install docker-ce \
        --install docker-ce-cli \
        --install containerd.io \
        --install docker-buildx-plugin \
        --install docker-compose-plugin \
        --install fail2ban \
        --install fail2ban-firewalld \
        --install policycoreutils-python-utils
else
    rpm-ostree install \
        docker-ce docker-ce-cli containerd.io \
        docker-buildx-plugin docker-compose-plugin \
```

```
        fail2ban fail2ban-firewalld \
        policycoreutils-python-utils
fi
systemctl reboot
```

Package	Purpose
docker-ce	The Docker Engine — the core daemon that runs containers.
docker-ce-cli	The docker command-line tool you type commands into.
containerd.io	The container runtime Docker uses under the hood.
docker-buildx-plugin	Extended build capabilities for Docker images.
docker-compose-plugin	Enables the docker compose command for multi-container apps.
fail2ban	Monitors log files and automatically bans IPs showing brute-force signs.
fail2ban-firewalld	The Fail2ban action module that integrates with firewalld.
policycoreutils-python-utils	Provides the semanage command for SELinux port label management.

WARNING

Your SSH session will disconnect. systemctl reboot terminates the SSH connection immediately with "broken pipe." This is expected. Wait ~60 seconds, then reconnect.

PITFALL

Do not use ujust install-docker. This Bazzite built-in shortcut has a confirmed bug (GitHub #2130): the Docker service fails to start after install and docker-compose is not included. The rpm-ostree method above is the correct path.

PHASE 4

Ironclad Security

SELinux Port · SSH Keys · firewalld · Fail2ban

CONCEPT — WHAT IS SELINUX?

SELinux (Security-Enhanced Linux) is a mandatory access control system built into the Linux kernel. Unlike traditional permissions, SELinux attaches security "labels" to every file, process, and network port. A process can only access resources with compatible labels. It runs in enforcing mode on Bazzite — violations are blocked, not just logged.

1 Label port 22022 in the SELinux policy

Server — MUST BE FIRST

CRITICAL

Why SELinux must come first: SELinux controls which ports sshd is permitted to bind to. Only port 22 has the ssh_port_t label by default. If we restart sshd on port 22022 without adding this label first, SELinux will silently block the bind syscall — sshd will appear to start successfully but will not actually listen on any port, locking you out permanently.

```
# Add the label. If re-running after a partial setup, use -m (modify) instead of -a (add):
sudo semanage port -a -t ssh_port_t -p tcp 22022
# If you get "already defined", run: sudo semanage port -m -t ssh_port_t -p tcp 22022

sudo semanage port -l | grep ssh
# Expected: ssh_port_t    tcp    22022, 22
```

2 Generate and copy SSH keys

Personal Computer

CONCEPT — WHAT ARE SSH KEYS?

SSH key authentication uses cryptography instead of passwords. You generate a key pair: a private key (stays on your computer, never shared) and a public key (copied to the server). An attacker who intercepts network traffic gets nothing useful — there is no password to guess or intercept. After this phase, password authentication is permanently disabled.

Linux / macOS / WSL:

```
ssh-keygen -t ed25519   # Skip if you already have ~/.ssh/id_ed25519
ssh-copy-id username@192.168.1.50
ssh username@192.168.1.50
# Expected: you log in without being prompted for a password.
```

Windows (PowerShell):

```
ssh-keygen -t ed25519
scp $env:USERPROFILE\.ssh\id_ed25519.pub username@192.168.1.50:~/id_ed25519.pub
ssh username@192.168.1.50 "mkdir -p ~/.ssh && cat ~/id_ed25519.pub >> ~/.ssh/authorized_keys && rm
~/id_ed25519.pub"
ssh username@192.168.1.50
```

CRITICAL

Do not proceed until key login is confirmed working. The next step permanently disables password authentication. If your key is not in place, you will be locked out with no recovery path except physical console access.

3 Create the SSH hardening drop-in config

Server

CONCEPT — WHY USE A DROP-IN FILE?

On Bazzite (an immutable OS), OS updates can modify or replace /etc/ssh/sshd_config. Files placed in /etc/ssh/sshd_config.d/ are never touched by OS updates. The 99-hardening.conf filename sorts last alphabetically, ensuring our settings override any defaults.

```
# This heredoc uses no quotes on EOF, so $(whoami) is expanded right now,
# inserting your actual username into the AllowUsers line automatically.
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf > /dev/null << EOF
Port 22022
PubkeyAuthentication yes
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
LoginGraceTime 20
AllowUsers $(whoami)
EOF
```

Directive	Effect
Port 22022	Moves SSH off port 22. Eliminates 99%+ of automated bot traffic.
PubkeyAuthentication yes	Explicitly enables public key authentication.
PasswordAuthentication no	Permanently disables password login. Only your SSH key can authenticate.
PermitRootLogin no	Blocks direct login as root, even with a valid key.
MaxAuthTries 3	Allows only 3 authentication attempts per connection before dropping it.
LoginGraceTime 20	Closes the connection if authentication is not completed within 20 seconds.
AllowUsers your_name	Explicit allowlist — only your account can log in remotely.

```
sudo sshd -t   # Validate config syntax. No output = no errors.
# DO NOT restart sshd yet — open port 22022 in the firewall first.
```

4 Configure firewalld and restart sshd in the correct order

CRITICAL

Order is critical: (1) allow port 22022, (2) restart sshd, (3) verify connection on 22022, (4) only then close port 22. Never close port 22 before confirming port 22022 works.

```
# Step 4a — Open port 22022 BEFORE restarting sshd.
sudo firewall-cmd --permanent --add-port=22022/tcp
sudo firewall-cmd --reload
```

```
# Step 4b — NOW restart sshd.
sudo systemctl restart sshd
```

CRITICAL

Step 4c — TEST NOW. Open a NEW terminal on your personal computer and connect: ssh -p 22022 username@192.168.1.50 — Expected: successful login. Do not proceed until this succeeds. Keep your existing SSH session open as a safety net.

```
# Step 4d — Remove port 22 ONLY AFTER confirming port 22022 works.
sudo firewall-cmd --permanent --remove-service=ssh
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
# Expected: ports: 22022/tcp — port 22 must NOT appear.
```

5 Configure and enable Fail2ban

CONCEPT — WHAT IS FAIL2BAN?

Even with key-only authentication, attackers can still flood your server with connection attempts. Fail2ban monitors SSH connection logs in real time. If an IP address makes too many failed authentication attempts within a time window, Fail2ban automatically adds a firewall ban rule blocking that IP entirely.

```
sudo tee /etc/fail2ban/jail.local > /dev/null << 'EOF'
[sshd]
enabled = true
port = 22022
ignoreip = 127.0.0.1/8 ::1 192.168.0.0/16 10.0.0.0/8 172.16.0.0/12
backend = systemd
banaction = firewallcmd-rich-rules
EOF
```

Option	Effect
[sshd]	Jail name — Fail2ban knows how to parse sshd log patterns for this jail type.
enabled = true	Activate this jail.
port = 22022	Ban IPs on port 22022, not the default 22.
ignoreip	Never ban these IP ranges. Covers localhost and all three RFC 1918 private ranges.
backend = systemd	Fedora/Bazzite logs to the systemd journal. Tells Fail2ban to read from journalctl.
banaction = firewallcmd-rich-rules	Use firewalld rich rules to implement bans. Works for IPv4 and IPv6 on Bazzite.

```
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
# Expected: Port list: 22022
```

PHASE 5

Docker Initialisation

All remaining steps run on the server via SSH on port 22022

1 Enable Docker and add your user to the docker group

```
ssh -p 22022 username@192.168.1.50
sudo systemctl enable --now docker
grep -q "^docker:" /etc/group || sudo sh -c "getent group docker >> /etc/group"
sudo usermod -aG docker $USER
exit
ssh -p 22022 username@192.168.1.50
```

SECURITY

The docker group equals effective root. Any user in the docker group can start a container that mounts the entire host filesystem. This is an accepted trade-off on a single-user server where SSH access is locked to your key only. Guard your SSH private key accordingly.

2 Prevent Docker logs from filling your disk

CONCEPT — WHY CONFIGURE LOG ROTATION?

By default, Docker containers write logs indefinitely to JSON files. A verbose container like Nextcloud AIO can generate gigabytes of logs over months, eventually filling the OS drive and causing crashes. We configure a global policy: each container gets a maximum of 3 log files, each capped at 10 MB — totalling ~30 MB per container.

```
sudo tee /etc/docker/daemon.json > /dev/null << 'EOF'
{
    "log-driver": "json-file",
    "log-opts": { "max-size": "10m", "max-file": "3" }
}
EOF
sudo systemctl restart docker
```

3 Create the shared Docker network

CONCEPT — WHAT IS A DOCKER NETWORK?

By default, each Docker container is isolated on its own private network. We create a shared network called nextcloud-aio so that Nginx Proxy Manager (NPM) and Nextcloud AIO can talk to each other internally using container names as hostnames.

```
docker network create nextcloud-aio
```

PHASE 6

Data Drive Setup

CRITICAL

Complete this before starting any Nextcloud containers. The NEXTCLOUD_DATADIR path must exist, be mounted, and have the correct SELinux label before AIO's first start. It cannot be changed afterward without a full data migration.

PITFALL

Ordering is critical: Mount the filesystem FIRST, then apply the SELinux label. Running restorecon before mounting labels the OS drive's empty mount-point inode. When the ext4 partition mounts over it, that inode is replaced — the label is lost.

1 Format, mount, then label the data drive

```
lsblk
DATA_DRIVE="/dev/nvmeXnY"   # <-- REPLACE THIS with your actual data drive
echo "Target drive: $DATA_DRIVE" && lsblk "$DATA_DRIVE"

sudo parted "$DATA_DRIVE" --script mklabel gpt
sudo parted "$DATA_DRIVE" --script mkpart primary ext4 1MiB 100%
sudo partprobe "$DATA_DRIVE"
sleep 2
DATA_PART=$(lsblk -rn -o NAME "$DATA_DRIVE" | tail -n 1 | awk '{print "/dev/"$1}')
echo "Partition detected: $DATA_PART"
sudo mkfs.ext4 "$DATA_PART"
```

WARNING

This will ERASE ALL DATA on the target drive. Confirm the correct drive with lsblk before proceeding.

CONCEPT — WHAT IS CHATTR +I AND WHY USE IT HERE?

The +i flag sets the "immutable" attribute: not even root can create, delete, or write files inside an immutable directory. We set this on the empty /srv/nextcloud-data directory while the data drive is unmounted. If the drive ever fails to mount at boot, Docker gets a "permission denied" error and the container safely crashes instead of re-initialising a blank database or corrupting the OS drive. IMPORTANT: apply chattr +i BEFORE mounting the drive.

```
sudo mkdir -p /srv/nextcloud-data
sudo chattr +i /srv/nextcloud-data
lsattr -d /srv/nextcloud-data
# Expected: ----i-------------- /srv/nextcloud-data

PART_UUID=$(sudo blkid -s UUID -o value "$DATA_PART")
echo "Captured UUID: $PART_UUID"
echo "UUID=${PART_UUID} /srv/nextcloud-data ext4 defaults,nofail 0 2"
echo "UUID=${PART_UUID} /srv/nextcloud-data ext4 defaults,nofail 0 2" \
    | sudo tee -a /etc/fstab
sudo findmnt --verify
sudo mount -a
df -h /srv/nextcloud-data
# Expected: shows the data drive's size — NOT the OS drive's size.
```

CONCEPT — WHY USE /VAR/SRV INSTEAD OF /SRV FOR THE SELINUX RULE?

On Fedora Atomic, /srv is a symbolic link that points to /var/srv. SELinux's semanage fcontext stores file context rules literally. If we store the rule as /srv/..., SELinux labels the path with var_t rather than container_file_t. Nextcloud containers run with the container_t domain and cannot write to var_t paths. Using /var/srv ensures the correct label is applied.

```
sudo semanage fcontext -a -t container_file_t "/var/srv/nextcloud-data(/.*)?"
sudo restorecon -Rv /var/srv/nextcloud-data
ls -Zd /srv/nextcloud-data
# Expected: system_u:object_r:container_file_t:s0 /srv/nextcloud-data
```

NOTE

Why semanage fcontext instead of chcon? chcon labels files directly but these labels are reset if the filesystem is ever relabelled during Fedora Atomic OS updates. semanage fcontext writes the rule into SELinux's persistent policy database and re-applies the correct label automatically after every update.

PHASE 7

Deploying Services

DuckDNS · Nginx Proxy Manager · Nextcloud AIO

CONCEPT — WHAT IS A DOCKER COMPOSE FILE?

docker-compose.yml is a YAML configuration file that describes one or more containers as a single unit. YAML is indentation-sensitive — 2 spaces per level. Never use tabs; always use spaces.

1 DuckDNS auto-updater

WARNING

Clock check before proceeding: timedatectl | grep "Time zone" and timedatectl | grep "RTC in local TZ". Expected: RTC in local TZ: no. If "yes": sudo timedatectl set-local-rtc 0

```
mkdir -p ~/docker/duckdns && cd ~/docker/duckdns
TZ_VALUE=$(timedatectl show -p Timezone --value)
nano .env
# Contents:
# DUCKDNS_SUBDOMAINS=yourname   (subdomain only, no .duckdns.org)
# DUCKDNS_TOKEN=your-token-here
chmod 600 .env

cat > docker-compose.yml << EOF
services:
  duckdns:
    image: lscr.io/linuxserver/duckdns:latest
    container_name: duckdns
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    env_file:
      - .env
    environment:
      - SUBDOMAINS=\${DUCKDNS_SUBDOMAINS}
      - TOKEN=\${DUCKDNS_TOKEN}
      - TZ=${TZ_VALUE}
EOF
docker compose up -d
```

2 Nginx Proxy Manager (NPM)

CONCEPT — WHAT IS A REVERSE PROXY?

A reverse proxy sits in front of your services and routes incoming requests. When a visitor connects to yourname.duckdns.org, their request arrives at NPM (on port 443). NPM looks at the domain name, finds the matching proxy host rule, and forwards the request internally to the correct container. NPM also handles TLS termination: it obtains and renews SSL certificates from Let's Encrypt.

```
mkdir -p ~/docker/npm && cd ~/docker/npm
mkdir -p data letsencrypt
sudo chown -R $USER:$USER ./data ./letsencrypt

cat > docker-compose.yml << 'EOF'
services:
```

```
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    container_name: nginx-proxy-manager
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    ports:
      - '80:80'              # HTTP and Let's Encrypt challenges.
      - '443:443'            # HTTPS.
      - '127.0.0.1:81:81'   # Admin panel — localhost only.
    networks:
      - nextcloud-aio
    volumes:
      - ./data:/data:z
      - ./letsencrypt:/etc/letsencrypt:z
networks:
  nextcloud-aio:
    external: true
EOF
docker compose up -d
```

3 Nextcloud All-in-One

CONCEPT — WHAT IS NEXTCLOUD AIO?

Nextcloud AIO is a special "mastercontainer" that manages all the other containers Nextcloud needs: the main app server, database, caching layer, and cron job runner. The mastercontainer connects to the Docker socket to create, start, and stop child containers on your behalf. This is why native Docker is required.

WARNING

AMD GPU hardware acceleration: If you want VA-API GPU acceleration for Nextcloud video transcoding, do NOT run docker compose up -d below yet. See Appendix C first.

```
mkdir -p ~/docker/nextcloud && cd ~/docker/nextcloud
cat > docker-compose.yml << 'EOF'
volumes:
  nextcloud_aio_mastercontainer:
    name: nextcloud_aio_mastercontainer
services:
  nextcloud-aio-mastercontainer:
    image: ghcr.io/nextcloud-releases/all-in-one:latest
    init: true
    restart: always
    container_name: nextcloud-aio-mastercontainer
    security_opt:
      - label:disable   # REQUIRED: allows access to the Docker socket under SELinux.
    labels:
      - "com.centurylinklabs.watchtower.enable=false"   # Never auto-update AIO.
    volumes:
      - nextcloud_aio_mastercontainer:/mnt/docker-aio-config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports:
      - '127.0.0.1:8080:8080'   # AIO admin panel. Localhost only.
    networks:
      - nextcloud-aio
    environment:
      - APACHE_PORT=11000
      - APACHE_IP_BINDING=127.0.0.1
      - NEXTCLOUD_DATADIR=/srv/nextcloud-data
      # Cannot be changed after first start without a full data migration.
      # Uncomment ONLY if your router lacks NAT loopback (see Phase 10 Step 3):
      # - SKIP_DOMAIN_VALIDATION=true
networks:
  nextcloud-aio:
    external: true
```

```
EOF
docker compose up -d
```

PHASE 8

Session Management Scripts

start-kde · start-gaming · stop-session

CONCEPT — THE CORE PROBLEM THIS PHASE SOLVES

When you SSH in and run a command, your SSH connection is a "session" tracked by systemd-logind. Everything you start from that SSH session lives in the same "cgroup." When you disconnect, logind's KillUserProcesses=yes sends SIGTERM to everything in that cgroup. systemd-run creates a transient system-level service that lives outside your SSH session's cgroup, immune to cleanup.

CONCEPT — HOW THE SDDM ONE-SHOT PATTERN WORKS

(1) Write a temporary autologin config pointing to the desired session, (2) restart SDDM which reads the config synchronously and launches the session, (3) schedule a systemd-run transient timer to delete the launch config and restore the headless override 10 seconds later.

1 Write start-kde.sh

CONCEPT — WHY $(LOGNAME) INSTEAD OF $USER?

$USER changes to "root" inside a sudo shell. logname reads your original login name directly from the kernel's login record — it always returns your actual account name.

```
sudo tee /usr/local/bin/start-kde.sh > /dev/null << 'EOF'
#!/bin/bash
# start-kde.sh — Launch a KDE Wayland session on the physical display.
SDDM_USER="yourusername"
KDE_SESSION="plasma"
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"
log() { echo "[start-kde $(date '+%H:%M:%S')] $*"; }
# 1. Cancel stale timers
while IFS= read -r unit; do
    log "Cancelling stale timer: $unit"; systemctl stop "$unit" 2>/dev/null
done < <(systemctl list-units --type=timer --no-legend 2>/dev/null | awk '{print $1}' | grep
'^restore-headless-')
# 2. Guard: refuse if seat0 user session is already active
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null | awk '{print $1}' | while read -r sid; do
    seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) || continue
    state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) || continue
    class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
    [[ "$class" != "user" ]] && continue
    [[ "$seat" == "seat0" ]] && [[ "$state" == "active" || "$state" == "online" ]] && { echo
"$sid"; break; }
done)
if [[ -n "$ACTIVE" ]]; then
    log "ERROR: A graphical session ($ACTIVE) is already active on seat0."; exit 1
fi
# 3. Write one-shot autologin config
rm -f "$HEADLESS_CONF"; mkdir -p /etc/sddm.conf.d
cat > "$LAUNCH_CONF" << CFGEOF
[Autologin]
User=${SDDM_USER}
Session=${KDE_SESSION}
Relogin=false
CFGEOF
log "SDDM autologin config written for KDE Wayland."
# 4. Restart SDDM
systemctl reset-failed sddm 2>/dev/null; systemctl restart sddm
log "KDE Wayland session launching on physical display."
```

```
# 5. Schedule headless restoration as system-level transient service (immune to cgroup cleanup)
systemd-run --no-ask-password --collect \
    --unit="restore-headless-$(date +%s)" --on-active=10 \
    /bin/bash -c 'rm -f /etc/sddm.conf.d/zz-session-launch.conf && \
    printf "[Autologin]\nUser=\nSession=\nRelogin=false\n" \
    > /etc/sddm.conf.d/zzz-headless-server.conf'
log "Headless config restoration scheduled in 10 s."
EOF
sudo chmod 755 /usr/local/bin/start-kde.sh
sudo sed -i "s/yourusername/$(logname)/" /usr/local/bin/start-kde.sh
grep "SDDM_USER=" /usr/local/bin/start-kde.sh
# Expected: SDDM_USER="your_actual_username
```

2 Write start-gaming.sh

```
sudo tee /usr/local/bin/start-gaming.sh > /dev/null << 'EOF'
#!/bin/bash
# start-gaming.sh — Launch Gaming Mode on the physical display.
SDDM_USER="yourusername"
GAMING_SESSION="gamescope-session-steam"
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"
log() { echo "[start-gaming $(date '+%H:%M:%S')] $*"; }
# 1. Cancel stale timers
while IFS= read -r unit; do
    log "Cancelling stale timer: $unit"; systemctl stop "$unit" 2>/dev/null
done < <(systemctl list-units --type=timer --no-legend 2>/dev/null | awk '{print $1}' | grep
'^restore-headless-')
# 2. Guard: refuse if seat0 user session is already active
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null | awk '{print $1}' | while read -r sid; do
    seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) || continue
    state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) || continue
    class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
    [[ "$class" != "user" ]] && continue
    [[ "$seat" == "seat0" ]] && [[ "$state" == "active" || "$state" == "online" ]] && { echo
"$sid"; break; }
done)
if [[ -n "$ACTIVE" ]]; then
    log "ERROR: A session ($ACTIVE) is already active on seat0."; exit 1
fi
# 3. Verify session file exists
SESSION_FILE="/usr/share/wayland-sessions/${GAMING_SESSION}.desktop"
if [[ ! -f "$SESSION_FILE" ]]; then
    log "ERROR: Session file not found: $SESSION_FILE"
    log "Available sessions:"; ls /usr/share/wayland-sessions/ | sed 's/^/  /'; exit 1
fi
# 4. Write one-shot autologin config
rm -f "$HEADLESS_CONF"; mkdir -p /etc/sddm.conf.d
cat > "$LAUNCH_CONF" << CFGEOF
[Autologin]
User=${SDDM_USER}
Session=${GAMING_SESSION}
Relogin=false
CFGEOF
# 5. Restart SDDM
systemctl reset-failed sddm 2>/dev/null; systemctl restart sddm
log "Gaming Mode launching on physical display."
# 6. Schedule headless restoration
systemd-run --no-ask-password --collect \
    --unit="restore-headless-$(date +%s)" --on-active=10 \
    /bin/bash -c 'rm -f /etc/sddm.conf.d/zz-session-launch.conf && \
    printf "[Autologin]\nUser=\nSession=\nRelogin=false\n" \
    > /etc/sddm.conf.d/zzz-headless-server.conf'
log "Headless config restoration scheduled in 10 s."
EOF
sudo chmod 755 /usr/local/bin/start-gaming.sh
```

```
sudo sed -i "s/yourusername/$(logname)/" /usr/local/bin/start-gaming.sh
grep "SDDM_USER=" /usr/local/bin/start-gaming.sh
```

3 Write stop-session.sh

Emergency fallback — terminates whichever graphical session is active on seat0. Polls until the SDDM greeter appears on seat0 before returning.

```
sudo tee /usr/local/bin/stop-session.sh > /dev/null << 'EOF'
#!/bin/bash
# stop-session.sh — Terminate the active seat0 session and return to headless idle.
LAUNCH_CONF="/etc/sddm.conf.d/zz-session-launch.conf"
HEADLESS_CONF="/etc/sddm.conf.d/zzz-headless-server.conf"
log() { echo "[stop-session $(date '+%H:%M:%S')] $*"; }
# 1. Cancel stale timers
while IFS= read -r unit; do
    log "Cancelling stale timer: $unit"; systemctl stop "$unit" 2>/dev/null
done < <(systemctl list-units --type=timer --no-legend 2>/dev/null | awk '{print $1}' | grep
'^restore-headless-')
# 2. Write headless config immediately
rm -f "$LAUNCH_CONF"
printf '[Autologin]\nUser=\nSession=\nRelogin=false\n' > "$HEADLESS_CONF"
# 3. Find and terminate active seat0 user session
ACTIVE=$(loginctl list-sessions --no-legend 2>/dev/null | awk '{print $1}' | while read -r sid; do
    seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) || continue
    state=$(loginctl show-session "$sid" -p State --value 2>/dev/null) || continue
    class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
    [[ "$class" != "user" ]] && continue
    [[ "$seat" == "seat0" ]] && [[ "$state" == "active" || "$state" == "online" ]] && { echo
"$sid"; break; }
done)
if [[ -n "$ACTIVE" ]]; then
    log "Terminating session $ACTIVE on seat0... (WARNING: unsaved work is lost)"
    loginctl terminate-session "$ACTIVE"; sleep 3
else
    log "No active seat0 user session found."
fi
# 4. Restart SDDM to reclaim DRM master
systemctl reset-failed sddm 2>/dev/null; systemctl restart sddm
# 5. Poll for SDDM greeter (GPU/DRM handover confirmation)
GREETER_READY=false
for i in $(seq 1 25); do
    sleep 1
    GREETER=$(loginctl list-sessions --no-legend 2>/dev/null | awk '{print $1}' | while read -r
sid; do
        seat=$(loginctl show-session "$sid" -p Seat --value 2>/dev/null) || continue
        class=$(loginctl show-session "$sid" -p Class --value 2>/dev/null) || continue
        [[ "$class" == "greeter" && "$seat" == "seat0" ]] && { echo "$sid"; break; }
    done)
    if [[ -n "$GREETER" ]]; then
        log "SDDM greeter ready after ${i}s."; GREETER_READY=true; sleep 3; break
    fi
done
[[ "$GREETER_READY" != "true" ]] && log "Warning: SDDM greeter not detected within 25s."
log "Done. SDDM login screen should be on the physical display."
EOF
sudo chmod 755 /usr/local/bin/stop-session.sh
```

4 Configure sudo — session scripts, health monitor, and Fail2ban

All passwordless sudo entries are consolidated into a single drop-in file. This covers the session scripts, the server-health dashboard, and the fail2ban-client query used by the dashboard.

```
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/start-kde.sh\n" "$USER" \
    | sudo tee /etc/sudoers.d/session-scripts > /dev/null
```

```
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/start-gaming.sh\n" "$USER" \
    | sudo tee -a /etc/sudoers.d/session-scripts > /dev/null
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/stop-session.sh\n" "$USER" \
    | sudo tee -a /etc/sudoers.d/session-scripts > /dev/null
printf "%s ALL=(ALL) NOPASSWD: /usr/local/bin/server-health\n" "$USER" \
    | sudo tee -a /etc/sudoers.d/session-scripts > /dev/null
printf "%s ALL=(ALL) NOPASSWD: /usr/bin/fail2ban-client\n" "$USER" \
    | sudo tee -a /etc/sudoers.d/session-scripts > /dev/null

sudo chmod 440 /etc/sudoers.d/session-scripts
sudo visudo -c -f /etc/sudoers.d/session-scripts
# Expected: /etc/sudoers.d/session-scripts: parsed OK
```

NOTE

server-health is installed in Appendix E. Adding it to sudoers here keeps all privilege grants in one place. The script will not exist yet — that is expected and does not affect the sudoers file validity.

5 Test the scripts manually

```
sudo start-kde.sh        # KDE appears on the display.
sudo start-gaming.sh     # Must ERROR — session already active.
# Log out of KDE via Application Menu → Leave → Log Out. Then:
sudo start-gaming.sh     # Gaming Mode launches.
sudo stop-session.sh     # Blocks ~5–15 s, then "Done."

ls /usr/share/wayland-sessions/
# Must include: gamescope-session-steam.desktop and plasma.desktop
ls -la /usr/local/bin/start-kde.sh /usr/local/bin/start-gaming.sh /usr/local/bin/stop-session.sh
# Expected: -rwxr-xr-x 1 root root ... for all three
```

PHASE 9

WireGuard VPN + Sunshine/Moonlight Streaming

Zero-trust VPN · Hardware-accelerated streaming · No RDP required

CONCEPT — WHY WIREGUARD + SUNSHINE INSTEAD OF KRDP/RDP?

krdp uses TCP, which suffers from "TCP Meltdown" during packet loss — stuttering and freezing that makes gaming unplayable. Sunshine uses UDP with VA-API hardware encoding, delivering 4K/60fps with near-zero latency. WireGuard (also UDP) provides the secure tunnel: unauthenticated packets are silently dropped, making port 51820 invisible to portscanning tools. No RDP client required — only the WireGuard and Moonlight apps on your device.

SECURITY

Port 51820/UDP is the only new internet-facing port. Sunshine ports are opened in firewalld — accessible from your local home network directly. They are not accessible from the internet because they are NEVER forwarded at the router. All remote streaming travels exclusively through the encrypted WireGuard tunnel.

1 Host OS Preparation

Server — Three substeps

Step 1a — Load legacy NAT kernel modules.

```
sudo modprobe iptable_nat iptable_mangle
printf "iptable_nat\niptable_mangle\n" | sudo tee /etc/modules-load.d/iptables-legacy.conf
lsmod | grep iptable
# Expected: iptable_nat and iptable_mangle both listed.
```

Step 1b — Grant hardware permissions and set up Sunshine.

```
# Add your user to input (keyboard/mouse capture), video (GPU display),
# and render (VA-API hardware encoding via /dev/dri/renderD128) groups.
sudo usermod -aG input,video,render $(whoami)

# Bazzite built-in Sunshine setup: installs Sunshine (if not present),
# configures udev rules for input device access, and applies SELinux contexts.
sudo ujust setup-sunshine
```

Step 1c — Enable login persistence (linger) and reboot.

```
# loginctl enable-linger: allows systemd user services (like sunshine.service)
# to run at boot without an active interactive login session.
sudo loginctl enable-linger $(whoami)

# Group changes and linger REQUIRE a full reboot to take effect.
sudo systemctl reboot
# Wait ~60 seconds, then reconnect:
ssh -p 22022 username@192.168.1.50
```

CRITICAL

Pitfall #16: if you skip the reboot and try to start Sunshine immediately, it will fail silently. The usermod group changes and linger setting are not effective until a new login session is created after reboot.

2 Deploy wg-easy (WireGuard VPN)

Router prerequisite: Forward port 51820/UDP to your server's local IP (e.g., 192.168.1.50) in your router's admin panel. This is the only router change for Phase 9.

Step 2a — Generate the admin password hash.

```
# Run the wg-easy hash utility via Docker (no extra packages needed).
# Replace 'your-admin-password' with a strong password.
docker run --rm ghcr.io/wg-easy/wg-easy wgpw 'your-admin-password'
# Output: $2a$12$xxxxxxxxx...
# IMPORTANT: In docker-compose.yml, every $ in this hash must be doubled ($$).
# Example: $2a$12$abc becomes $$2a$$12$$abc
```

Step 2b — Create the wg-easy compose file.

```
mkdir -p ~/docker/wireguard && cd ~/docker/wireguard
cat > docker-compose.yml << 'EOF'
services:
  wg-easy:
    image: ghcr.io/wg-easy/wg-easy:latest
    container_name: wg-easy
    restart: unless-stopped
    labels:
      - "com.centurylinklabs.watchtower.enable=true"
    environment:
      - WG_HOST=yourname.duckdns.org    # Your public DuckDNS domain.
      - PASSWORD_HASH=$$2a$$12$$...     # Paste bcrypt hash ($ doubled). No quotes.
    volumes:
      - ./data:/etc/wireguard:Z          # :Z = SELinux volume label for Fedora Atomic.
    ports:
      - "51820:51820/udp"                # WireGuard VPN port (Docker nftables injection).
      - "127.0.0.1:51821:51821/tcp"      # Admin web UI — localhost only (SSH tunnel below).
    cap_add:
      - NET_ADMIN       # Required to create the wg0 interface and configure routes.
      - SYS_MODULE      # Safety net for kernel module loading in container namespace.
    sysctls:
      - net.ipv4.ip_forward=1                    # Required for VPN client routing.
      - net.ipv4.conf.all.src_valid_mark=1        # Required for WireGuard source-based routing.
EOF
docker compose up -d
docker ps | grep wg-easy
# Expected: wg-easy ... Up ... 0.0.0.0:51820->51820/udp, 127.0.0.1:51821->51821/tcp
```

Step 2c — Access the admin panel and create a client profile.

```
# On your PERSONAL COMPUTER — open an SSH tunnel to the wg-easy admin panel.
ssh -N -o ServerAliveInterval=60 -L 51821:127.0.0.1:51821 -p 22022 username@your-server-ip
# Open http://localhost:51821 in your browser.
# Log in. Click "+ New Client". Name it (e.g., "laptop").
# Download the .conf file or scan the QR code with the WireGuard app.
# The server's VPN IP will be 10.8.0.1 (wg-easy default: 10.8.0.0/24).
```

INFO

Split tunnel (recommended): Edit the downloaded WireGuard .conf file on your device. Change AllowedIPs = 0.0.0.0/0 to AllowedIPs = 10.8.0.0/24. This routes only VPN-subnet traffic through the tunnel. Your regular internet stays on your local connection — only traffic to the server (10.8.0.1) goes through the VPN. Lower latency, higher throughput for normal browsing.

3 Configure Sunshine streaming

Step 3a — Open Sunshine firewall ports.

```
sudo firewall-cmd --permanent --add-port=47984/tcp   # Sunshine control
sudo firewall-cmd --permanent --add-port=47989/tcp   # Sunshine HTTPS control
sudo firewall-cmd --permanent --add-port=48010/tcp   # Sunshine RTSP
sudo firewall-cmd --permanent --add-port=47998/udp   # Sunshine video
sudo firewall-cmd --permanent --add-port=47999/udp   # Sunshine control (UDP)
sudo firewall-cmd --permanent --add-port=48000/udp   # Sunshine audio
sudo firewall-cmd --permanent --add-port=48002/udp   # Sunshine input
sudo firewall-cmd --permanent --add-port=48010/udp   # Sunshine RTSP (UDP)

# CRITICAL: after every firewall-cmd --reload, Docker's nftables rules are wiped.
# Always restart Docker immediately after any firewalld reload (Pitfall #17).
sudo firewall-cmd --reload
sudo systemctl restart docker
sudo firewall-cmd --list-ports
# Expected: 22022/tcp and all Sunshine ports listed above.
```

Step 3b — Enable the Sunshine user service.

```
systemctl --user enable --now sunshine
systemctl --user status sunshine
# Expected: Active: active (running)
```

Step 3c — Initial Sunshine web UI setup.

```
# On your PERSONAL COMPUTER — SSH tunnel to the Sunshine admin panel.
ssh -N -o ServerAliveInterval=60 -L 47990:127.0.0.1:47990 -p 22022 username@your-server-ip
# Open https://localhost:47990 in your browser.
# Accept the self-signed certificate warning (Sunshine's own admin panel cert).
# Set a username and password for the Sunshine admin panel on first run.
```

INFO

Sunshine auto-detects the current desktop. When KDE is running (via start-kde.sh), Moonlight shows a "Desktop" stream. When gamescope/Gaming Mode is running, Steam Big Picture streams directly.

4 Remote workflow — client side

Step	Action
1. Connect VPN	Open the WireGuard app and activate your profile. The server becomes reachable at 10.8.0.1.
2. Launch session	ssh -p 22022 username@your-server-ip 'sudo start-kde.sh' (or start-gaming.sh for Steam Big Picture).
3. Open Moonlight	Open Moonlight on your device. Click "+ Add Host" and enter 10.8.0.1. Moonlight will discover Sunshine automatically.
4. Stream	Select "Desktop" (KDE) or "Steam Big Picture" (Gaming Mode) from the Moonlight app and connect.
5. Stop streaming	Close Moonlight. The session continues on the server display until you log out.

WARNING

Moonlight may struggle to find the server if your DNS resolves the server hostname to its LAN IP while the VPN is active. Always connect to Moonlight using the explicit VPN IP 10.8.0.1 — not the hostname.

PHASE 10

Link & Initialise Nextcloud

NPM config · Nextcloud setup

1 Access admin panels via SSH tunnel

The admin panels are bound to 127.0.0.1 (server's localhost) — invisible to your LAN. Create SSH tunnels from your personal computer and leave them running while you use the panels.

```
# NPM admin panel — available at http://localhost:8081 in your browser
ssh -N -o ServerAliveInterval=60 -L 8081:127.0.0.1:81 -p 22022 username@192.168.1.50
# Nextcloud AIO setup panel — available at https://localhost:9443 in your browser
ssh -N -o ServerAliveInterval=60 -L 9443:127.0.0.1:8080 -p 22022 username@192.168.1.50
# Run each tunnel in a separate terminal window. Leave both open.
```

2 Configure Nginx Proxy Manager

INFO

First boot delay: Start the NPM tunnel, then wait 30–60 seconds. On its very first boot, NPM generates its SQLite database and internal SSL keys. Opening the panel too early returns a 502 Bad Gateway — wait and retry. Once ready, open http://localhost:8081. Log in with the default credentials admin@example.com / changeme and change both immediately.

Go to Hosts → Proxy Hosts → Add Proxy Host:

WARNING

CRITICAL: Do NOT click Save until you have configured the Details, SSL, and Advanced tabs in the same session. If you save after only filling in the Details tab, NPM silently resets the SSL tab to defaults on the next edit — leaving you with a working proxy but no TLS and no error message. Complete all three tabs and click Save once.

Field	Value
Domain Names	yourname.duckdns.org — the public domain NPM will respond to
Scheme	http — NPM communicates with the Nextcloud app container internally over plain HTTP; encryption happens at NPM's edge
Forward Hostname / IP	nextcloud-aio-apache — the Docker container name. On the shared network, container names resolve as DNS hostnames. Use the name, not an IP.
Forward Port	11000 — the port AIO's app server listens on (set via APACHE_PORT in Phase 7)
Cache Assets	OFF — do NOT enable. Nextcloud manages its own caching headers; NPM's asset caching conflicts with them and breaks authentication.
Block Common Exploits	ON — adds basic WAF rules blocking common web attack patterns
Websockets Support	ON — required for Nextcloud's real-time collaboration features

SSL tab: Request new certificate · Force SSL · HTTP/2 · Agree to ToS → Save

INFO

HSTS (recommended, after confirming HTTPS is working): Edit the proxy host again and enable HSTS in the SSL tab. Leave "Also apply to subdomains" and "HSTS Preload" OFF unless you have a specific reason to use them. HSTS instructs browsers to always use HTTPS for your domain and must only be enabled after the certificate is confirmed valid.

Required: Advanced tab — large file uploads. After saving the proxy host, click Edit → Advanced tab and paste:

```
client_max_body_size 0;
proxy_read_timeout 36000s;
proxy_send_timeout 36000s;
proxy_request_buffering off;
proxy_hide_header Upgrade;
```

Directive	Effect
client_max_body_size 0	Removes NPM's default upload size limit. Without this, files larger than ~1 MB fail with HTTP 413.
proxy_read/send_timeout 36000s	Allows 10-hour timeouts for large file uploads over slow connections.
proxy_request_buffering off	Sends the request body directly to the backend. Required for Nextcloud's chunked upload protocol.
proxy_hide_header Upgrade	Prevents mobile clients (iOS, Android) from receiving "Connection Closed" errors during sync. Required per Nextcloud server documentation.

INFO

Expect a 502 error when testing the proxy host in NPM. The nextcloud-aio-apache container does not exist until the AIO wizard (Step 4) downloads and starts it. A 502 at this stage means NPM is configured correctly. This is expected.

3 Fix local network access (NAT loopback)

CONCEPT — WHAT IS NAT LOOPBACK / HAIRPINNING?

When you access yourname.duckdns.org from inside your own home network, your DNS lookup returns your public IP address. Your device then tries to connect to that public IP — but the packet never leaves your home network. Many consumer routers cannot handle this "loopback." The solution is to make your domain resolve to the server's local IP when accessed from inside the home network.

Solutions — try in order:

Solution 1 — Router local DNS override (recommended): In your router's admin panel, look for "Local DNS", "DNS Entries", or "Static Hosts". Add: yourname.duckdns.org → 192.168.1.50.

Solution 2 — Pi-hole local DNS: If you use Pi-hole, add a "Local DNS Record" entry: yourname.duckdns.org → 192.168.1.50.

Solution 3 — SKIP_DOMAIN_VALIDATION (last resort):

```
cd ~/docker/nextcloud
# Edit docker-compose.yml: uncomment the SKIP_DOMAIN_VALIDATION line.
docker compose down && docker compose up -d
```

INFO

SKIP_DOMAIN_VALIDATION=true bypasses AIO's domain reachability check at setup time only. This has no effect on security after setup. Editing your client's /etc/hosts does NOT solve this. The domain validation request originates from inside the AIO Docker container on the server itself — host file changes on your PC are irrelevant to that check.

4 Initialise Nextcloud AIO

Stop the NPM tunnel (Ctrl+C). Start the Nextcloud AIO tunnel. Go to https://localhost:9443. Your browser will show a security warning about a self-signed certificate — this is expected (it's the AIO admin panel's own certificate, not your public one). Accept and continue. Copy the setup password shown, enter your domain (yourname.duckdns.org), and proceed to the container selection screen.

Optional Container	Why
Nextcloud Hub version	Always select latest (Hub 26+). Includes the most recent features and security patches.
Fulltextsearch	Optional — requires 16 GB+ RAM. Indexes file contents for full-text search. Skip on 8 GB systems.
HaRP	Recommended. Modern replacement for the older Talk socket proxy. Lower resource usage.
Whiteboard / Imaginary	Optional, low impact. Collaborative whiteboard and image preview generation. Can be added later.

Click **Start Containers** and wait 5–10 minutes for all containers to download and initialise.

INFO

No second drive yet for backups? The AIO wizard shows a Borg backup configuration screen before you can log in. If you don't have a dedicated backup drive yet, click "Setup without a backup solution" to proceed now. You can configure Borg later (see Phase 11).

PHASE 11

Testing & Backups

1 DNS & External Tests

```
# Query Google's DNS (8.8.8.8) directly — bypasses local DNS, verifies external record.
# Result must be your home's public IP address.
nslookup yourname.duckdns.org 8.8.8.8

# If nslookup is not available:
ping -c 1 yourname.duckdns.org
```

External Test: Disable Wi-Fi on your phone and visit https://yourname.duckdns.org over cellular data. A green padlock and the Nextcloud login screen confirms the full chain: DuckDNS → public IP → router port forwarding → NPM → Nextcloud AIO → Let's Encrypt certificate.

2 Session Management Test

```
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'
# KDE appears on the physical display. SSH terminal returns promptly.
ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# Must output ERROR — KDE is still active.
# Log out of KDE via Application Menu → Leave → Log Out. Then:
ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# Gaming Mode appears on the physical display.
ssh -p 22022 username@your-server-ip 'sudo stop-session.sh'
# Blocks ~5–15 s, then: "Done. SDDM login screen should be on the physical display."
```

3 Streaming Test

```
# 1. Connect WireGuard on your client device.
# 2. Verify VPN connectivity:
ping 10.8.0.1
# Expected: replies from the server.
# 3. Launch KDE on the server:
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'
# 4. Open Moonlight, connect to 10.8.0.1, stream Desktop.
# Expected: smooth stream with hardware-accelerated encoding.
```

4 System State Validation Checklist

Run these after completing all phases. Every check has a specific expected result.

Group A — OS, SSH & Security Layer

```
ls /etc/sddm.conf.d/           # Expected: zzz-headless-server.conf only.
loginctl list-sessions          # Expected: empty or SSH session only — no seat0 entry.
ss -tlnp | grep sshd            # Expected: exactly one line showing *:22022. No *:22.
sudo semanage port -l | grep ssh # Expected: ssh_port_t tcp 22022, 22
systemctl status sleep.target suspend.target hibernate.target \
    hybrid-sleep.target suspend-then-hibernate.target | grep -E 'Loaded:|●'
```

```
# Expected: all five lines show "masked".
sudo fail2ban-client status sshd # Expected: Port list: 22022
```

Group B — Storage, Docker & Services

```
ls -Zd /srv/nextcloud-data
# Expected: system_u:object_r:container_file_t:s0 /srv/nextcloud-data
grep nextcloud-data /etc/fstab   # Expected: line contains "nofail".
lsattr -d /srv/nextcloud-data    # Expected: ----i-------------- /srv/nextcloud-data
docker info 2>&1 | grep 'Server Version'  # Expected: no "permission denied".
docker ps --format 'table {{.Names}}■{{.Status}}'
# Expected: duckdns, nginx-proxy-manager, nextcloud-aio-mastercontainer, wg-easy — all "Up".
sudo firewall-cmd --list-all
# Expected: ports: 22022/tcp plus Sunshine ports. 80/443/51820 managed by Docker — absent is
correct.
curl -sI http://localhost:81 | head -1   # Expected: HTTP/1.1 302 Found
```

Group C — Session Scripts, Streaming & Cgroup Escape

```
ls -la /usr/local/bin/start-kde.sh /usr/local/bin/start-gaming.sh \
    /usr/local/bin/stop-session.sh /usr/local/bin/server-health
# Expected: -rwxr-xr-x 1 root root ... for all four.
ssh root@127.0.0.1 -p 22022    # Expected: Permission denied (publickey)
rpm-ostree status               # Expected: docker-ce under LayeredPackages.
systemctl --user status sunshine # Expected: active (running).
docker ps | grep wg-easy        # Expected: wg-easy Up.
# Cgroup escape test:
sudo start-kde.sh && exit       # Launch KDE and immediately disconnect.
# Wait 10 seconds, then reconnect:
ssh -p 22022 username@your-server-ip
loginctl list-sessions          # Expected: KDE session on seat0.
ls /etc/sddm.conf.d/            # Expected: zzz-headless-server.conf (timer ran after disconnect).
```

5 Backups

CONCEPT — WHAT IS BORG BACKUP?

Borg is free, open-source backup software. Your data never leaves your home. Live data lives on your SSD at /srv/nextcloud-data. Borg creates compressed, deduplicated snapshots of your live SSD data onto a second physical drive. If the SSD fails or you accidentally delete something, you restore from the Borg backup.

WARNING

Hardware required: A second physical drive dedicated to backups. Without it, Borg has nowhere to write. If you don't have one yet, click "Setup without a backup solution" in the AIO wizard and come back to this once you do.

Inside the AIO setup panel (https://localhost:9443 via SSH tunnel): find the Borg Backup section. Set a destination path on your second drive (e.g., /mnt/backup-drive) and schedule daily backups.

CRITICAL

Perform a test restore immediately after the first backup completes. A backup you have never tested cannot be trusted. The restore path is documented in the AIO panel. Perform a test restore once a month.

APPENDIX A

Troubleshooting

Machine boots to Gaming Mode after initial install

```
# SSH is not yet available on a fresh install.
# Go to Gaming Mode → Power menu → Switch to Desktop.
# Then run Phase 1 Steps 2–4 to enable SSH and apply the headless override.
```

start-kde.sh / start-gaming.sh reports "session already active" but nothing is visible

```
loginctl list-sessions
# Note the stuck session ID.
loginctl terminate-session SESSION_ID
sleep 2
sudo start-kde.sh   # or start-gaming.sh
```

SDDM shows login screen instead of autologging in

```
ls /usr/share/wayland-sessions/
# Expected: plasma.desktop and gamescope-session-steam.desktop
grep 'KDE_SESSION=' /usr/local/bin/start-kde.sh
grep 'GAMING_SESSION=' /usr/local/bin/start-gaming.sh
journalctl -u sddm -n 50 --no-pager
```

Sunshine not streaming / Moonlight cannot connect

```
# 1. Confirm WireGuard is connected: ping 10.8.0.1
# 2. Confirm KDE or Gaming Mode is running: loginctl list-sessions
# 3. Confirm Sunshine is running: systemctl --user status sunshine
# 4. Confirm Sunshine ports are open: sudo firewall-cmd --list-ports
# 5. On server, re-run: systemctl --user restart sunshine
```

Sunshine service fails to start

```
# Confirm linger is enabled:
loginctl show-user $USER | grep Linger
# Expected: Linger=yes
# If "no": sudo loginctl enable-linger $USER && sudo systemctl reboot
```

Docker containers not running after a reboot

```
sudo systemctl status docker
sudo systemctl enable --now docker
cd ~/docker/npm      && docker compose up -d
cd ~/docker/nextcloud && docker compose up -d
cd ~/docker/duckdns  && docker compose up -d
cd ~/docker/wireguard && docker compose up -d
```

zz-session-launch.conf persists (power loss during script run)

```
sudo rm -f /etc/sddm.conf.d/zz-session-launch.conf
sudo tee /etc/sddm.conf.d/zzz-headless-server.conf > /dev/null << 'EOF'
[Autologin]
User=
Session=
Relogin=false
EOF
ls /etc/sddm.conf.d/   # Expected: only zzz-headless-server.conf
```

Data drive not mounting — system dropped to emergency mode

This should not happen if nofail is in fstab (Phase 6).

```
# Connect a display and keyboard. At the emergency shell:
nano /etc/fstab
# Add "nofail" to the /srv/nextcloud-data options column.
systemctl reboot
```

firewall-cmd --reload takes Nextcloud offline

```
# Docker's nftables rules were wiped. Restore them:
sudo systemctl restart docker
# If Nextcloud was actively syncing, put it in maintenance mode first:
docker exec nextcloud-aio-nextcloud php occ maintenance:mode --on
# ...restart Docker, wait for containers to recover, then:
docker exec nextcloud-aio-nextcloud php occ maintenance:mode --off
```

APPENDIX B

Normal Workflows — Quick Reference

Commands to run from your personal computer

Use KDE on the TV or monitor

```
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'
# KDE Wayland session appears on the physical display.
# To stop: Application Menu → Leave → Log Out.
```

Stream the desktop remotely (Sunshine + Moonlight)

```
# 1. Connect WireGuard on your client device.
# 2. Launch KDE:
ssh -p 22022 username@your-server-ip 'sudo start-kde.sh'
# 3. Open Moonlight, connect to 10.8.0.1, select Desktop.
```

Stream Gaming Mode remotely

```
# 1. Connect WireGuard on your client device.
# 2. If KDE is running, log out first via KDE's logout button. Then:
ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# 3. In Moonlight, select Steam Big Picture.
```

Game locally (physical display only)

```
# If KDE is running, log out first. Then:
ssh -p 22022 username@your-server-ip 'sudo start-gaming.sh'
# To stop: Power menu in Steam Big Picture → Exit to Desktop.
```

Force-stop a stuck session

```
ssh -p 22022 username@your-server-ip 'sudo stop-session.sh'
# WARNING: all unsaved work in the terminated session is lost.
# Script blocks ~5–15 s polling for the SDDM greeter, then returns.
```

Check what is currently running

```
loginctl list-sessions
# seat0 entry present = a graphical session is active on the physical display.
ls /etc/sddm.conf.d/
# Healthy state: only zzz-headless-server.conf is present.
```

CRITICAL

Never run firewall-cmd --reload while Docker is running without immediately running sudo systemctl restart docker afterward. firewalld --reload flushes and rebuilds the entire nftables ruleset. Docker's port-forwarding rules are wiped. Consequence: ports 80, 443, and 51820/UDP become unreachable with no error message.

APPENDIX C

Optional: AMD GPU Hardware Acceleration for Nextcloud

AMD and Intel GPUs only · Complete before first docker compose up -d in Phase 7

WARNING

AMD and Intel GPUs only. Do not follow this appendix on NVIDIA systems. Timing requirement: This configuration must be applied BEFORE running docker compose up -d for the first time in Phase 7 Step 3. If you have already started the containers and want to add hardware acceleration, you must stop all containers (docker compose down), remove the AIO data volume (docker volume rm nextcloud_aio_mastercontainer), and start fresh. This does not delete your Nextcloud user data, but does reset the AIO configuration.

CONCEPT — WHAT IS VA-API?

VA-API (Video Acceleration API) is a Linux interface for hardware-accelerated video encode and decode operations. Instead of the CPU processing every video frame in software, VA-API lets applications offload that work to the GPU's dedicated hardware encoder/decoder. For AMD GPUs, VA-API is provided by the Mesa radeonsi driver, which is included in Bazzite.

CONCEPT — WHAT IS /DEV/DRI?

The DRI (Direct Rendering Infrastructure) subsystem exposes the GPU to userspace applications as device files in /dev/dri/. There are two types: cardX files (used by compositors for display output) and renderDX files (used by applications for GPU compute and hardware transcoding). Nextcloud needs access to renderD128 for VA-API transcode operations.

1 Verify your AMD GPU and render nodes are present

```
ls -la /dev/dri/
# Expected: card0 (display output) and renderD128 (render node — what Nextcloud needs)
vainfo 2>/dev/null | head -20
# Expected: lists supported VA-API profiles (H.264, HEVC, AV1, etc.)
# If vainfo is not installed: sudo rpm-ostree install libva-utils && systemctl reboot
```

2 Remove the immutable guard

BEFORE first container start

The AIO mastercontainer's first-start initialisation writes ownership data to the data directory. The chattr +i immutable flag blocks this.

```
df -h /srv/nextcloud-data   # Confirm the data drive is mounted first.
sudo chattr -i /srv/nextcloud-data
lsattr -d /srv/nextcloud-data
# Expected: ------------------ /srv/nextcloud-data (no "i" in any position)
```

3 Update the Nextcloud AIO compose file

```
cd ~/docker/nextcloud
cat > docker-compose.yml << 'EOF'
volumes:
  nextcloud_aio_mastercontainer:
    name: nextcloud_aio_mastercontainer
services:
  nextcloud-aio-mastercontainer:
```

```
    image: ghcr.io/nextcloud-releases/all-in-one:latest
    init: true
    restart: always
    container_name: nextcloud-aio-mastercontainer
    security_opt:
      - label:disable
    labels:
      - "com.centurylinklabs.watchtower.enable=false"
    devices:
      - /dev/dri:/dev/dri   # Passes GPU render node into the container for VA-API.
    volumes:
      - nextcloud_aio_mastercontainer:/mnt/docker-aio-config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports:
      - '127.0.0.1:8080:8080'
    networks:
      - nextcloud-aio
    environment:
      - APACHE_PORT=11000
      - APACHE_IP_BINDING=127.0.0.1
      - NEXTCLOUD_DATADIR=/srv/nextcloud-data
      - NEXTCLOUD_ENABLE_DRI_DEVICE=true
      # Tells AIO to add the www-data user inside the Nextcloud app container
      # to the "render" group so it can access renderD128.
      # - SKIP_DOMAIN_VALIDATION=true
networks:
  nextcloud-aio:
    external: true
EOF
```

4 Start containers and re-apply the immutable guard

```
cd ~/docker/nextcloud && docker compose up -d
# Monitor container startup (5–15 min on first run).
watch docker ps --format 'table {{.Names}}■{{.Status}}'
# Wait until all AIO containers show "(healthy)".

# Re-apply the immutable flag immediately after all containers show healthy.
sudo chattr +i /srv/nextcloud-data
lsattr -d /srv/nextcloud-data
# Expected: ----i-------------- /srv/nextcloud-data (the "i" is back)
```

CRITICAL

Do not skip re-applying the immutable flag. Once all containers show healthy, immediately re-apply chattr +i. This restores blind-mount protection.

5 Verify hardware acceleration is active

```
docker exec nextcloud-aio-nextcloud ffmpeg -hwaccels 2>/dev/null | grep -i vaapi
# Expected: "vaapi" listed as an available hardware accelerator.
```

After verifying GPU acceleration works, proceed to Phase 10 to configure NPM and initialise Nextcloud. GPU acceleration works automatically — no additional Nextcloud configuration is required.

APPENDIX D

OS Updates, Rollbacks & Container Updates

OS Updates & Rollbacks

Bazzite stages OS updates silently in the background. Applying them requires a reboot — during which your server is offline for 1–2 minutes. All containers come back up automatically (restart: always / restart: unless-stopped).

```
# Roll back to the previous OS image instantly.
rpm-ostree rollback && systemctl reboot

# List all available stable builds by tag:
skopeo list-tags docker://ghcr.io/ublue-os/bazzite-deck \
    | grep -- 'stable-' | sort -rV

# Rebase to a specific build:
rpm-ostree rebase ostree-image-signed:docker://ghcr.io/ublue-os/bazzite-deck:stable-YEARMONTHDAY
```

WARNING

After a major OS rebase (e.g., Fedora 40 → 41): Docker CE is layered via rpm-ostree and a major version rebase may need the Docker layer reinstalled. If Docker fails to start after a rebase: rpm-ostree rollback && systemctl reboot, then re-follow Phase 3.

Watchtower — Auto-update Containers

CONCEPT — WHAT IS WATCHTOWER?

Watchtower is a Docker container that watches your other containers for new image versions. We use opt-in mode: only containers explicitly labelled with com.centurylinklabs.watchtower.enable=true are updated (DuckDNS, NPM, wg-easy). Nextcloud AIO containers are excluded because Nextcloud database migrations must be handled through the AIO panel, not by simply restarting with a new image.

```
mkdir -p ~/docker/watchtower && cd ~/docker/watchtower
TZ_VALUE=$(timedatectl show -p Timezone --value)
cat > docker-compose.yml << EOF
services:
  watchtower:
    image: nickfedor/watchtower
    # nickfedor/watchtower is the actively maintained fork.
    # containrrr/watchtower was archived December 2025; its Docker client library
    # is locked to API v1.25 which Docker Engine v29+ rejects with a hard error.
    restart: unless-stopped
    security_opt:
      - label:disable   # Required for Docker socket access on SELinux systems.
    environment:
      - TZ=${TZ_VALUE}
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    command: --label-enable --cleanup --schedule "0 0 4 * * *"
    # ■ --label-enable: only update containers with watchtower.enable=true label.
    # ■ --cleanup: delete the old image after updating to free disk space.
    # ■ --schedule "0 0 4 * * *": run at 04:00 every day.
    # Format: second minute hour day-of-month month day-of-week.
EOF
docker compose up -d
```

INFO

Watchtower does not watch itself. Update Watchtower manually on a monthly basis: cd ~/docker/watchtower && docker compose pull && docker compose up -d

Nextcloud AIO — Manual Update Procedure

CRITICAL

Nextcloud AIO containers are intentionally excluded from Watchtower. The ~10 AIO child containers must be updated through the AIO admin panel at https://localhost:9443 (over SSH tunnel) every few weeks. Never update AIO containers by restarting them with a new image directly — this skips database migrations and can corrupt your data.

```
ssh -N -o ServerAliveInterval=60 -L 9443:127.0.0.1:8080 -p 22022 username@192.168.1.50
# Go to https://localhost:9443. Click Update when available.
# AIO handles all migrations and restarts in the correct order.
docker ps --format 'table {{.Names}}■{{.Status}}'
# Expected: all nextcloud-aio-* containers show "(healthy)".
```

APPENDIX E

Server Health Monitor

Live terminal dashboard · Run as a regular user via SSH

The server-health script is a live terminal dashboard that displays CPU, RAM, GPU, storage, Docker container status, Nextcloud health, SSL certificate expiry, Fail2ban ban count, WAN IP, and SDDM session state — all updating every 2 seconds.

1 Install the script

The script lives in /usr/local/bin/ alongside the session management scripts. Passwordless sudo access — needed to query fail2ban-client — is already configured in the /etc/sudoers.d/session-scripts drop-in written in Phase 8 Step 4. No additional sudoers configuration is required here.

```
sudo tee /usr/local/bin/server-health > /dev/null << 'SCRIPTEOF'
#!/bin/bash
# server-health — Bazzite server status dashboard
# Live mode: updates every few seconds · Ctrl+C to exit

R=$'\033[0m'; DIM=$'\033[2;37m'; BLD=$'\033[1m'
CYN=$'\033[1;36m'; BLU=$'\033[1;34m'; WHT=$'\033[1;37m'
GRN=$'\033[1;32m'; YEL=$'\033[1;33m'; RED=$'\033[1;31m'

THICK="${CYN}$(printf '━%.0s' {1..72})${R}"
THIN="${DIM}$(printf '─%.0s' {1..72})${R}"
REFRESH=2      # seconds between redraws
SLOW_EVERY=4   # cycles between slow-data refresh (every 8 s at REFRESH=2)

row()      { printf "  ${BLU}%-8s${R}  %b\n" "$1" "${*:2}"; }
dot_ok()   { printf "${GRN}●${R}"; }
dot_warn() { printf "${YEL}●${R}"; }
dot_err()  { printf "${RED}●${R}"; }
dot_dim()  { printf "${DIM}●${R}"; }

# Colour-coded dot based on percentage thresholds
pct_dot() {
    local p=$1 warn=${2:-70} err=${3:-90}
    if   (( p >= err  )); then dot_err
    elif (( p >= warn )); then dot_warn
    else dot_ok; fi
}

# ── Cleanup on exit ──────────────────────────────────────────────────────
trap 'tput cnorm; printf "\n"; exit 0' INT TERM
[ -t 1 ] && tput civis   # hide cursor while live

# ── GPU detection (AMD = 0x1002, Intel = 0x8086) ────────────────────────
GPU_DEV=""
for _card in /sys/class/drm/card*/; do
    _vendor=$(cat "${_card}device/vendor" 2>/dev/null)
    [[ "$_vendor" == "0x1002" || "$_vendor" == "0x8086" ]] && \
        { GPU_DEV="${_card}device"; break; }
done

GPU_HWMON=""
if [ -n "$GPU_DEV" ]; then
    GPU_HWMON=$(find "${GPU_DEV}/hwmon" -maxdepth 2 -name "temp1_input" 2>/dev/null \
        | head -1 | xargs -I{} dirname {} 2>/dev/null)
fi

# ── CPU baseline sample ──────────────────────────────────────────────────
_cpustat() { awk '/^cpu /{print $2,$3,$4,$5,$6,$7,$8; exit}' /proc/stat; }
prev_cs=$(_cpustat)
cpu_pct=0
```

```
# ── CPU Power (RAPL) init ──────────────────────────────────────────────
RAPL_FILE="/sys/class/powercap/intel-rapl:0/energy_uj"
prev_energy=""
prev_time=$(date +%s%3N)
[ -r "$RAPL_FILE" ] && prev_energy=$(cat "$RAPL_FILE" 2>/dev/null)
cpu_power="calc..."

# ── Slow-cache defaults (shown until first background refresh) ─────────
WAN_IP="–"; F2B="–"
NPM_DOT=$(dot_dim); NPM_ST="–"
NC_DOT=$(dot_dim);  NC_ST="–"
CERT_DOT=$(dot_dim); CERT_ST="–"
BK_DOT=$(dot_dim);  BK_ST="–"

refresh_slow() {
    # WAN IP
    WAN_IP=$(curl -s --max-time 4 https://api.ipify.org 2>/dev/null)
    [ -z "$WAN_IP" ] && WAN_IP="unreachable"

    # Fail2ban
    F2B=$(sudo fail2ban-client status sshd 2>/dev/null \
        | grep 'Currently banned' | grep -oE '[0-9]+')
    [ -z "$F2B" ] && F2B="?"

    # NPM
    _nc=$(curl -s -o /dev/null -w "%{http_code}" --max-time 2 http://localhost:81 2>/dev/null)
    [ "${_nc:-000}" != "000" ] && { NPM_DOT=$(dot_ok); NPM_ST="ok"; } \
                               || { NPM_DOT=$(dot_err); NPM_ST="down"; }

    # Nextcloud maintenance mode
    _nr=$(docker exec nextcloud-aio-nextcloud php occ maintenance:mode 2>/dev/null)
    if   echo "$_nr" | grep -qi "disabled\|not enabled"; then NC_DOT=$(dot_ok);   NC_ST="live"
    elif echo "$_nr" | grep -qi "enabled";               then NC_DOT=$(dot_warn);
NC_ST="maintenance"
    else NC_DOT=$(dot_dim); NC_ST="n/a"; fi

    # SSL cert
    _ce=$(docker exec nginx-proxy-manager openssl x509 -noout -enddate \
        -in /etc/letsencrypt/live/npm-1/cert.pem 2>/dev/null | cut -d= -f2)
    if [ -n "$_ce" ]; then
        _cd=$(( ( $(date -d "$_ce" +%s) - $(date +%s) ) / 86400 ))
        if   (( _cd <= 14 )); then CERT_DOT=$(dot_err);  CERT_ST="cert expires ${_cd}d"
        elif (( _cd <= 30 )); then CERT_DOT=$(dot_warn); CERT_ST="cert expires ${_cd}d"
        else                       CERT_DOT=$(dot_ok);   CERT_ST="cert valid ${_cd}d"
        fi
    else CERT_DOT=$(dot_warn); CERT_ST="cert pending"; fi

    # Backup log
    _bk=$(docker logs nextcloud-aio-mastercontainer 2>/dev/null \
        | grep -i "Backup finished successfully" | tail -1 | awk '{print $1,$2}')
    [ -n "$_bk" ] && { BK_DOT=$(dot_ok); BK_ST="$_bk"; } \
                  || { BK_DOT=$(dot_dim); BK_ST="no record found"; }
}

# ── Container block ────────────────────────────────────────────────────
print_containers() {
    local out
    out=$(docker ps -a --format '{{.Names}}#{{.Status}}' 2>/dev/null)
    if [ -z "$out" ]; then
        printf "  Docker not running or no containers found.\n"; return
    fi
    printf "%s\n" "$out" | awk -F'#' '{
        name=$1; st=$2
        if      (st~/healthy/)  { dot="\033[1;32m●\033[0m"; tag="healthy"  }
        else if (st~/^Up/)      { dot="\033[1;32m●\033[0m"; tag="up"       }
        else if (st~/starting/) { dot="\033[1;33m●\033[0m"; tag="starting" }
        else if (st~/^Exited/)  { dot="\033[1;31m●\033[0m"; tag="stopped"  }
        else                    { dot="\033[1;33m●\033[0m"; tag=substr(st,1,9) }
        entry=sprintf("  %-36s %s %-9s", name, dot, tag)
        if (NR%2==1) { row=entry }
        else         { printf "%s    %s\n", row, entry; row="" }
    } END { if (row!="") printf "%s\n", row }'
}
```

```
# ═══════════════════════════════════════════════════════════════════════════
# Main loop
# ═══════════════════════════════════════════════════════════════════════════
CYCLE=0

while true; do

    # ── CPU Power (RAPL) ──────────────────────────────────────────────────
    cur_time=$(date +%s%3N)
    elapsed_ms=$(( cur_time - prev_time ))

    if [ -n "$prev_energy" ]; then
        cur_energy=$(cat "$RAPL_FILE" 2>/dev/null)
        delta_uj=$(( cur_energy - prev_energy ))
        (( delta_uj < 0 )) && delta_uj=$(( cur_energy ))

        if (( elapsed_ms > 0 && delta_uj > 0 )); then
            # Convert microjoules to Watts: W = µJ / (ms * 100)  [×10 fixed-point]
            w_x10=$(( delta_uj / (elapsed_ms * 100) ))
            cpu_power="$(( w_x10 / 10 )).$(( w_x10 % 10 ))W"
        fi
        prev_energy=$cur_energy
    fi
    prev_time=$cur_time

    # ── CPU usage (delta since last sample) ───────────────────────────────
    cur_cs=$(_cpustat)
    read -ra _p <<< "$prev_cs"
    read -ra _c <<< "$cur_cs"
    _pt=0; _ct=0
    for v in "${_p[@]}"; do (( _pt += v )); done
    for v in "${_c[@]}"; do (( _ct += v )); done
    _dt=$(( _ct - _pt )); _di=$(( _c[3] - _p[3] ))
    (( _dt > 0 )) && cpu_pct=$(( 100 * (_dt - _di) / _dt )) || cpu_pct=0
    prev_cs="$cur_cs"

    # ── CPU clock (average across all online cores) ───────────────────────
    _fsum=0; _fn=0
    for _f in /sys/devices/system/cpu/cpufreq/policy*/scaling_cur_freq; do
        [ -r "$_f" ] || continue
        _fkhz=$(< "$_f")
        (( _fsum += _fkhz, _fn++ ))
    done
    if (( _fn > 0 )); then
        _mhz=$(( _fsum / _fn / 1000 ))
        if (( _mhz >= 1000 )); then
            cpu_freq="$(( _mhz / 1000 )).$(( (_mhz % 1000) / 100 ))GHz"
        else
            cpu_freq="${_mhz}MHz"
        fi
    else cpu_freq="n/a"; fi

    # ── CPU temp ──────────────────────────────────────────────────────────
    CPU_TEMP=$(sensors 2>/dev/null \
        | grep -E '(Tctl|Tdie|Package id 0)' | head -1 | awk '{print $2}')
    [ -z "$CPU_TEMP" ] && \
        CPU_TEMP=$(sensors 2>/dev/null | grep 'temp1' | head -1 | awk '{print $2}')
    [ -z "$CPU_TEMP" ] && CPU_TEMP="n/a"

    # ── Extra Temps (RAM & NVMe) ──────────────────────────────────────────
    RAM_TEMP=$(sensors 2>/dev/null | grep -A 2 'spd5118' | awk '/temp1/{print $2; exit}')
    NVME_TEMP=$(sensors 2>/dev/null | grep -A 2 'nvme-pci' | awk '/Composite/{print $2; exit}')

    # ── GPU ───────────────────────────────────────────────────────────────
    GPU_ROW=""
    if [ -n "$GPU_DEV" ]; then
        _gbusy=$(cat "${GPU_DEV}/gpu_busy_percent" 2>/dev/null)
        _gbusy=${_gbusy:-0}
        _gfhz=${GPU_HWMON:+$(cat "${GPU_HWMON}/freq1_input" 2>/dev/null)}
        if [ -n "$_gfhz" ] && (( _gfhz > 0 )); then
            gpu_freq="$(( _gfhz / 1000000 ))MHz"
        else gpu_freq="n/a"; fi
        _gt=${GPU_HWMON:+$(cat "${GPU_HWMON}/temp1_input" 2>/dev/null)}
        [ -n "$_gt" ] && gpu_temp="+$(( _gt / 1000 ))°C" || gpu_temp="n/a"
```

```
        _vu=$(cat "${GPU_DEV}/mem_info_vram_used"  2>/dev/null)
        _vt=$(cat "${GPU_DEV}/mem_info_vram_total" 2>/dev/null)
        if [ -n "$_vu" ] && [ -n "$_vt" ] && (( _vt > 0 )); then
            _vum=$(( _vu / 1048576 )); _vtm=$(( _vt / 1048576 ))
            if (( _vtm >= 1024 )); then
                _vui=$(( _vum / 1024 )); _vud=$(( (_vum % 1024) * 10 / 1024 ))
                _vti=$(( _vtm / 1024 )); _vtd=$(( (_vtm % 1024) * 10 / 1024 ))
                gpu_vram="${_vui}.${_vud}/${_vti}.${_vtd}GiB VRAM"
            else
                gpu_vram="${_vum}/${_vtm}MiB VRAM"
            fi
        else gpu_vram=""; fi
        _gpw=${GPU_HWMON:+$(cat "${GPU_HWMON}/power1_average" 2>/dev/null)}
        if [ -n "$_gpw" ] && (( _gpw > 0 )); then
            gpu_power="$(( _gpw / 1000000 )).$(( (_gpw % 1000000) / 100000 ))W"
        else gpu_power=""; fi
        GPU_ROW="$(pct_dot "$_gbusy" 70 90) ${_gbusy}%  @  ${gpu_freq}  ·  ${gpu_temp}"
        [ -n "$gpu_vram"  ] && GPU_ROW+="  ·  ${gpu_vram}"
        [ -n "$gpu_power" ] && GPU_ROW+="  ·  ${gpu_power}"
    fi

    # ── Memory ──────────────────────────────────────────────────────────
    RAM_USED=$(free -h | awk '/^Mem:/{print $3}')
    RAM_TOT=$( free -h | awk '/^Mem:/{print $2}')
    RAM_PCT=$( free    | awk '/^Mem:/{printf "%.0f",$3/$2*100}')
    SWAP_USED=$(free -h | awk '/^Swap:/{print $3}')
    SWAP_TOT=$( free -h | awk '/^Swap:/{print $2}')
    RAM_PCT=${RAM_PCT:-0}

    # ── Display ─────────────────────────────────────────────────────────
    SEAT0=$(loginctl list-sessions --no-legend 2>/dev/null | awk '$4=="seat0"{print $1;exit}')
    if [ -n "$SEAT0" ]; then
        CLS=$(loginctl show-session "$SEAT0" -p Class --value 2>/dev/null)
        if [ "$CLS" = "greeter" ]; then
            DISP_DOT=$(dot_ok); DISP_ST="idle  SDDM login screen"
        else
            DSK=$(loginctl show-session "$SEAT0" -p Desktop --value 2>/dev/null)
            DISP_DOT=$(dot_warn); DISP_ST="active  ${DSK:-session}"
        fi
    else
        DISP_DOT=$(dot_ok); DISP_ST="idle  no display active"
    fi
    if [ -f /etc/sddm.conf.d/zz-session-launch.conf ]; then
        SDDM_DOT=$(dot_warn); SDDM_ST="launch conf present"
    else
        SDDM_DOT=$(dot_ok); SDDM_ST="clean"
    fi

    # ── Storage ─────────────────────────────────────────────────────────
    OS_S=$(df -h /var 2>/dev/null | awk 'NR==2{print $3" / "$2" ("$5")"}')
    DAT_RAW=$(df -h /var/srv/nextcloud-data 2>/dev/null | awk 'NR==2{print $3" / "$2" ("$5")"}')
    if [ -n "$DAT_RAW" ]; then DAT_DOT=$(dot_ok); DAT_ST="$DAT_RAW"
    else DAT_DOT=$(dot_err); DAT_ST="not mounted"; fi

    # ── Network (fast parts) ────────────────────────────────────────────
    SSH_C=$(ss -tn state established 2>/dev/null | grep -c ":22022")

    # ── Systemd ─────────────────────────────────────────────────────────
    FAIL_N=$(systemctl --failed --no-legend 2>/dev/null | grep -c '.' || true)
    if [ "${FAIL_N:-0}" -gt 0 ]; then
        FAIL_NAMES=$(systemctl --failed --no-legend 2>/dev/null | awk '{printf "%s  ",$2}')
        SYS_DOT=$(dot_err); SYS_ST="${FAIL_N} failed  ${FAIL_NAMES}"
    else
        SYS_DOT=$(dot_ok); SYS_ST="healthy"
    fi

    # ── Slow refresh (network/docker — every SLOW_EVERY cycles) ─────────
    (( CYCLE > 0 && CYCLE % SLOW_EVERY == 0 )) && refresh_slow

    # ── Timestamps ──────────────────────────────────────────────────────
    NOW=$(date '+%a %d %b %Y  %H:%M:%S')
    HOST=$(hostname -s)
    UP=$(uptime -p | sed 's/up //')
    LOAD=$(awk '{print $1"  "$2"  "$3}' /proc/loadavg)
```

```
    # ── Paint ──────────────────────────────────────────────────────────────
    printf '\033[H\033[2J'
    printf "%s\n" "$THICK"
    printf "  BAZZITE  %-22s  %s\n" "$HOST" "$NOW"
    printf "%s\n" "$THICK"

    row "SYSTEM"   "up $UP  ·  load $LOAD"
    row "CPU"      "$(pct_dot "$cpu_pct" 60 85) ${cpu_pct}%  @  ${cpu_freq}  ·  ${CPU_TEMP}  ·  $
{cpu_power}"
    [ -n "$GPU_ROW" ] && row "GPU" "$GPU_ROW"
    row "MEMORY"   "$(pct_dot "$RAM_PCT" 75 90) RAM $RAM_USED / $RAM_TOT (${RAM_PCT}%)  ·  swap
$SWAP_USED / $SWAP_TOT${RAM_TEMP:+  ·  $RAM_TEMP}"
    row "DISPLAY"  "$DISP_DOT $DISP_ST  ·  sddm $SDDM_DOT $SDDM_ST"
    row "STORAGE"  "OS $OS_S  ·  data $DAT_DOT $DAT_ST${NVME_TEMP:+  ·  $NVME_TEMP}"
    row "NETWORK"  "SSH ${SSH_C} conn  ·  F2B ${F2B} banned  ·  WAN $WAN_IP"
    row "SYSTEMD"  "$SYS_DOT $SYS_ST"
    row "SERVICES" "NPM $NPM_DOT $NPM_ST  ·  Nextcloud $NC_DOT $NC_ST  ·  $CERT_DOT $CERT_ST"
    row "BACKUP"   "$BK_DOT $BK_ST"

    printf "\n%s\n" "$THIN"
    printf "  CONTAINERS\n"
    print_containers

    printf "\n%s\n" "$THICK"
    _ns=$(( (SLOW_EVERY - CYCLE % SLOW_EVERY) * REFRESH ))
    printf "  refresh ${REFRESH}s  ·  slow data in ${_ns}s  ·  journalctl -u sshd -f  ·  Ctrl+C to
exit\n"
    printf "%s\n" "$THICK"

    CYCLE=$(( CYCLE + 1 ))
    sleep "$REFRESH"

done
SCRIPTEOF
sudo chmod 755 /usr/local/bin/server-health
```

Verify installation and run:

```
ls -la /usr/local/bin/server-health
# Expected: -rwxr-xr-x 1 root root ...
server-health
```

2 What the dashboard monitors

Row	Source / Method
SYSTEM	uptime -p, /proc/loadavg
CPU	Delta from /proc/stat (usage%), /sys/devices/system/cpu/cpufreq/ (freq), lm-sensors (temp), RAPL energy counter (power draw — Intel CPUs only; shows n/a on AMD)
GPU	AMD: /sys/class/drm/cardX/device/ (usage, VRAM, power), hwmon sysfs (temp, freq). Supports AMD and Intel. Row omitted if no compatible GPU found.
MEMORY	free -h (RAM used/total/percent, swap), lm-sensors spd5118 (RAM temp if DDR5 with SPD hub)
DISPLAY	loginctl list-sessions (seat0 state, session Desktop value, greeter/user class)
STORAGE	df -h /var (OS drive), df -h /var/srv/nextcloud-data (data drive), lm-sensors nvme-pci (NVMe temp)
NETWORK	ss -tn (active SSH connections on port 22022), curl api.ipify.org (WAN IP, slow refresh)
SYSTEMD	systemctl --failed (count and names of failed units)
SERVICES	curl localhost:81 (NPM), docker exec nextcloud-aio-nextcloud php occ maintenance:mode (Nextcloud), openssl x509 (SSL cert expiry in days)
BACKUP	docker logs nextcloud-aio-mastercontainer (last successful Borg backup timestamp)
CONTAINERS	docker ps -a — all containers, status dot: green=healthy/up, yellow=starting, red=stopped. Two-column layout.

3 Refresh intervals

Fast data (CPU, GPU, RAM, storage, display, systemd) refreshes every 2 seconds. Slow data (WAN IP, Fail2ban count, NPM, Nextcloud, SSL cert, backup log) refreshes every 8 seconds. Both intervals are configurable at the top of the script via the REFRESH and SLOW_EVERY variables.

> **NOTE**
> The RAPL energy counter for CPU power draw (/sys/class/powercap/intel-rapl:0/) is Intel-specific. On AMD systems the CPU power row shows n/a — this is expected and harmless.